Neural Networks

A multilayer perceptron with backprop

The problem

- Linear models can't learn XOR-like patterns
- Hand-crafting features doesn't scale
- Want the model to learn its own features

Architecture



Forward pass

$$a_{l+1} = \mathrm{ReLU}(a_l W_l + b_l)$$

$$p = \mathrm{softmax}(a_{L-1} W_{L-1} + b_{L-1})$$

- He init: W ~ N(0, 2 / fan_in)
- Cache activations for the backward pass

Backpropagation

$$\delta_L = \frac{p - Y}{n}$$

$$\nabla W_l = {a_l}^\top \delta_l, \qquad \delta_{l-1} = (\delta_l {W_l}^\top) \odot [a_l > 0]$$

- Chain rule, layer by layer, from the output back

gradients()

```
def gradients(self, X, y):
    acts = self._forward(X)
    delta = acts[-1].copy()
    delta[np.arange(len(y)), y] -= 1
    delta /= len(y)
    gW, gb = [None] * len(self.W), [None] * len(self.b)
    for i in reversed(range(len(self.W))):
        gW[i] = acts[i].T @ delta
        gb[i] = delta.sum(0)
        if i > 0:
            delta = (delta @ self.W[i].T) * (acts[i] > 0)
    return gW, gb
```

Training loop

```
def fit(self, X, y):
    X, y = np.asarray(X, float), np.asarray(y)
    for _ in range(self.epochs):
        idx = self.rng.permutation(len(X))
        for s in range(0, len(X), self.batch_size):
            bi = idx[s:s + self.batch_size]
            gW, gb = self.gradients(X[bi], y[bi])
            for i in range(len(self.W)):
                self.W[i] -= self.lr * gW[i]
                self.b[i] -= self.lr * gb[i]
    return self
```

Complexity & trade-offs

- Forward/backward: $O(n \cdot \Sigma \text{ layer_in} \times \text{layer_out})$
- Universal approximators, but data hungry
- Needs tuning: lr, depth, width, regularisation
- Tools: dropout, batch norm, Adam, early stopping

Recap

- Layers: linear + non-linearity
- Softmax + cross-entropy → error = p − Y
- Backprop = chain rule backwards
- Train with mini-batch SGD